



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N.º 09

NOMBRE COMPLETO: Rosas Cañada Abraham

N.º de Cuenta: 318307866

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 03/05/2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Trabajo previo en Blender y captura de la pista

Antes de modificar el código del programa fue necesario realizar trabajo de preparación sobre los modelos en Blender. La nave original venía como una sola pieza, por lo que se separó en cinco mallas independientes para poder animarlas de forma jerárquica: el cuerpo principal, el ala izquierda, el ala derecha, la hélice izquierda y la hélice derecha. Cada pieza se exportó posteriormente como un archivo .obj independiente (nave.obj, ala.obj, ala2.obj, heliceizq.obj y heliceder.obj) para cargarlas como modelos separados dentro del programa.

Otro aspecto fundamental del trabajo en Blender consistió en ajustar el origen de cada pieza para garantizar que rotara sobre su propio eje. Al exportar un modelo cuyo origen permanece en el centro de la escena (0, 0, 0) y no en el centro geométrico de la pieza, las rotaciones aplicadas posteriormente en OpenGL hacen que el modelo describa círculos amplios alrededor de un punto que no le corresponde. Para evitar este comportamiento se seleccionó cada pieza por separado y se aplicó la operación Object → Set Origin → Origin to Geometry, lo que reposiciona el pivote justo en el centro de la malla. De esta manera, al rotar una hélice sobre el eje X gira sobre su propio centro como una hélice real, y al rotar un ala sobre el eje Y oscila desde su base sin desplazarse del cuerpo de la nave.

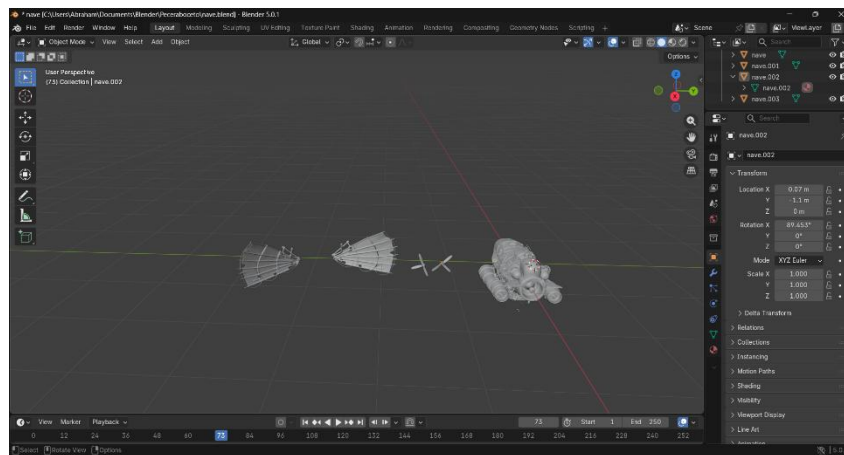
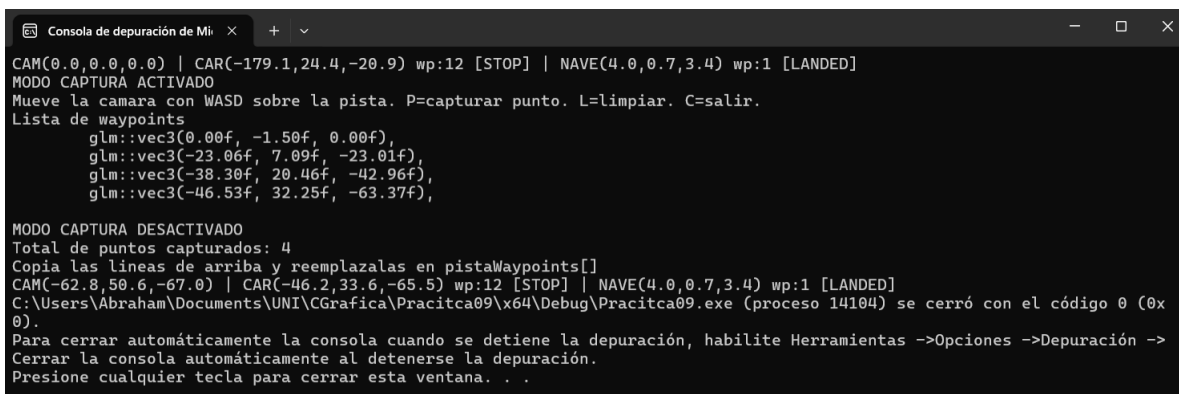


Figura 1. Vista en Blender de las piezas separadas de la nave (cuerpo, alas y hélices), con el origen de cada modelo previamente reposicionado mediante Set Origin para que cada pieza rote sobre su propio eje.

Adicionalmente, fue necesario obtener las coordenadas por las que pasa la pista. Dado que el modelo cuenta con curvas y rampas distribuidas a lo largo de aproximadamente 200 unidades, no resultaba viable definir manualmente las posiciones. Por esta razón se implementó dentro del programa un modo de captura interactivo: al presionar la tecla C, el carro se enlaza a la posición de la cámara y permite recorrer la pista en primera persona mediante las teclas WASD. Cada vez que se presiona la tecla P se imprime en consola la coordenada actual con el formato `glm::vec3(x, y, z)`, lista para insertarse directamente en el arreglo `pistaWaypoints`. La tecla L permite descartar la lista capturada y comenzar de nuevo, y al presionar nuevamente C se desactiva el modo de captura. Mediante este procedimiento se recolectaron los catorce puntos que definen el recorrido sobre la pista real.



```
Consola de depuración de Mi
CAM(0.0,0.0,0.0) | CAR(-179.1,24.4,-20.9) wp:12 [STOP] | NAVE(4.0,0.7,3.4) wp:1 [LANDED]
MODULO CAPTURA ACTIVADO
Mueve la camara con WASD sobre la pista. P=capturar punto. L=limpiar. C=salir.
Lista de waypoints
glm::vec3(0.00f, -1.50f, 0.00f),
glm::vec3(-23.06f, 7.09f, -23.01f),
glm::vec3(-38.30f, 20.46f, -42.96f),
glm::vec3(-46.53f, 32.25f, -63.37f),

MODULO CAPTURA DESACTIVADO
Total de puntos capturados: 4
Copia las líneas de arriba y reemplazalas en pistaWaypoints[]
CAM(-62.8,50.6,-67.0) | CAR(-46.2,33.6,-65.5) wp:12 [STOP] | NAVE(4.0,0.7,3.4) wp:1 [LANDED]
C:\Users\Abraham\Documents\UNI\CGrafica\Pracitca09\x64\Debug\Pracitca09.exe (proceso 14104) se cerró con el código 0 (0x0)
Para cerrar automáticamente la consola cuando se detiene la depuración, habilite Herramientas ->Opciones ->Depuración ->
Cerrar la consola automáticamente al detenerse la depuración.
Presione cualquier tecla para cerrar esta ventana. . .
```

Figura 2. Salida del modo de captura de waypoints en consola. Cada coordenada impresa corresponde a un punto de la pista registrado al presionar la tecla P, con el formato listo para insertarse directamente en el arreglo `pistaWaypoints`.

Como herramienta complementaria de depuración, durante la ejecución de la animación final el programa imprime en consola un tracker que muestra cada 0.1 segundos la posición actual de la cámara, del carro y de la nave, junto con el waypoint vigente y el estado del recorrido (GO, STOP, FLY, LANDING o LANDED). Este tracker resultó fundamental para verificar que cada objeto siguiera la trayectoria esperada y para diagnosticar errores cuando algún elemento no aparecía en pantalla.

Ejercicio 1: Hacer que el carro recorra la pista adecuadamente (curvas, inclinación en rampa, faro frontal ilumina la pista) desde el inicio hasta el punto extremo y ahí se detenga. Se puede repetir la animación si se presiona una tecla designada.

El recorrido del carro se construyó sobre el arreglo de catorce waypoints capturados previamente. El carro avanza interpolando linealmente entre el waypoint actual y el siguiente mediante una variable `carT` que recorre el rango de 0 a 1, y al alcanzar 1 avanza al siguiente segmento de la trayectoria. La velocidad de avance se controla con el parámetro `carVelAvance` multiplicado por `deltaTime`, lo que asegura que el movimiento sea consistente independientemente del framerate.

Para que el carro adopte la dirección correcta en las curvas, el ángulo de orientación (yaw) se calcula con la función `atan2(dirSeg.x, dirSeg.z)` sobre el vector del segmento actual. Esta operación devuelve el ángulo natural hacia donde apunta la siguiente porción de pista. Como una transición instantánea entre segmentos produciría un giro brusco, el yaw se interpola con la función `InterpolateYaw`, la cual además contempla el wrap-around del rango -180 a 180 grados para que el carro siempre realice el giro por el lado más corto.

Para la inclinación en rampa, el ángulo de pitch se obtiene a partir de la componente Y del segmento dividida entre la componente horizontal: `carPitchObjetivo = atan2(dirSeg.y, horizontal)`. De esta forma, si la rampa asciende dos unidades en una distancia horizontal de diez, el carro se inclina aproximadamente once grados hacia arriba, simulando visualmente la subida. La inclinación también se interpola mediante `InterpolateSuave` para que el cambio sea gradual y no instantáneo.

El faro frontal del carro está implementado mediante un `SpotLight (spotLights[2])` que se actualiza en cada frame para que su posición y dirección sigan al vehículo. La posición se calcula a 2.3 unidades por delante del carro y a 1.65 de altura, mientras que la dirección se obtiene a partir del vector horizontal del carro al cual se le suma una componente Y negativa de -0.25 para que el cono de luz incida sobre la pista justo enfrente del vehículo. El método `setFlash` se encarga de aplicar estos valores a la luz en tiempo de ejecución.

Cuando el carro alcanza el último waypoint del arreglo, la bandera `carDetenido` se activa y la animación se detiene de manera definitiva. Para reiniciar el recorrido se utiliza la tecla R, cuyo evento se detecta mediante un sistema de pulso único (se

compara el estado actual de la tecla con el estado del frame anterior, garantizando que cada pulsación se procese una sola vez). Al detectarse la pulsación, se invoca la función `ResetCarro()`, la cual restablece todas las variables de estado al primer waypoint: posición inicial, ángulo de orientación, ángulo de inclinación, rotación de las llantas y bandera de detenido.

Variables y arreglo de waypoints:

```
// Ruta de la pista
std::vector<glm::vec3> pistaWaypoints = {
    glm::vec3(3.99f, -0.84f, 3.39f),
    glm::vec3(-19.69f, -0.73f, 3.96f),
    // ... (14 waypoints en total)
    glm::vec3(-179.15f, 24.40f, -20.92f)
};

// Estado del carro
int carWpActual = 0;
float carT = 0.0f;
glm::vec3 carPos = pistaWaypoints[0];
float carYaw = 0.0f;
float carPitch = 0.0f;
bool carDetenido = false;
float rotllanta = 0.0f;
```

Lógica de avance, giro e inclinación del carro:

```
if (!carDetenido && pistaWaypoints.size() >= 2)
{
    glm::vec3 wpDesde = pistaWaypoints[carWpActual];
    glm::vec3 wpHacia = pistaWaypoints[carWpActual + 1];
    glm::vec3 segmento = wpHacia - wpDesde;
    float longitudSeg = glm::length(segmento);

    carT += (carVelAvance * deltaTime) / longitudSeg;

    if (carT >= 1.0f)
    {
        carT = 0.0f;
        carWpActual++;
        if (carWpActual >= (int)pistaWaypoints.size() - 1)
            carDetenido = true;
    }

    carPos = wpDesde + segmento * carT;
    glm::vec3 dirSeg = glm::normalize(segmento);

    // Yaw segun direccion del segmento (curvas)
    carYawObjetivo = atan2(dirSeg.x, dirSeg.z) / toRadians;
    carYaw = InterpolarYaw(carYaw, carYawObjetivo, carVelGiro, deltaTime);

    // Llantas girando
    rotllanta += carVelLlantas * deltaTime;
```

```

// Inclinacion en rampa
float horizontal = sqrt(dirSeg.x*dirSeg.x + dirSeg.z*dirSeg.z);
carPitchObjetivo = atan2(dirSeg.y, horizontal) / toRadians;
carPitch = InterpolarSuave(carPitch, carPitchObjetivo, carVelInclinacion,
deltaTime);
}

```

Faro frontal del carro y reinicio con tecla R:

```

// Faro frontal del carro
glm::vec3 dirCarroH = glm::vec3(carDir.x, 0.0f, carDir.z);
dirCarroH = glm::normalize(dirCarroH);

glm::vec3 posFaroCarro = carPos + glm::vec3(0.0f, 1.65f, 0.0f) + dirCarroH * 2.3f;
glm::vec3 dirFaroCarro = glm::normalize(dirCarroH + glm::vec3(0.0f, -0.25f, 0.0f));
spotLights[2].SetFlash(posFaroCarro, dirFaroCarro);

// Reset del carro con tecla R
bool teclaRActual = mainWindow.getKeys()[GLFW_KEY_R];
if (teclaRActual && !teclaRAnterior)
    ResetCarro();
teclaRAnterior = teclaRActual;

```

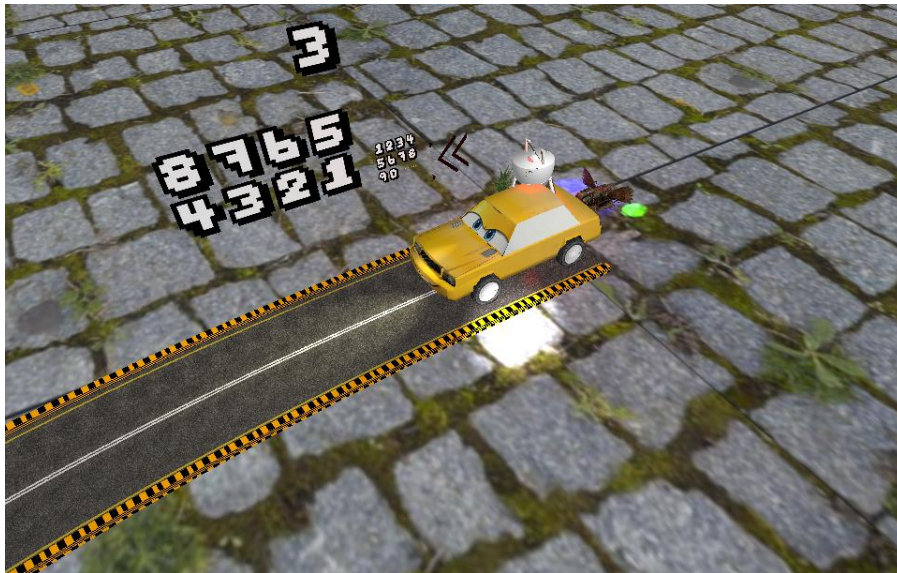


Figura 3. Carro al inicio de la pista, posicionado en el primer waypoint del recorrido.



Figura 4. Carro tomando una curva de la pista. La orientación se actualiza automáticamente según la dirección del segmento entre los waypoints actuales.



Figura 5. Carro inclinado al subir la rampa con el faro frontal proyectando luz sobre la pista. La inclinación se calcula a partir de la pendiente del segmento.



Figura 6. Carro detenido en el último waypoint del recorrido. La animación se detiene de forma definitiva hasta que se presiona la tecla R para reiniciarla.

Ejercicio 2: Separar de la nave la otra ala y las dos hélices, hacer que sobrevuele la pista adecuadamente (jerarquía, aleteo, giro de hélices, faro ilumina la pista) pero en sentido contrario, inicia en el punto extremo y aterriza junto al punto inicial. Esta animación no se puede repetir.

La nave utiliza el mismo arreglo pistaWaypoints que el carro, pero recorrido en orden inverso. La variable naveWpActual se inicializa en pistaWaypoints.size() - 1, correspondiente al último punto, y va decrementando segmento a segmento, de modo que en cada iteración la nave avanza desde pistaWaypoints[naveWpActual] hacia pistaWaypoints[naveWpActual - 1]. Este planteamiento cumple con el requisito de viajar en sentido contrario al recorrido del carro.

La nave sobrevuela la pista a una altura constante: a la posición sobre el segmento se le suma el valor naveAlturaVuelo (20 unidades en el eje Y) para que se desplace por encima del trazado sin tocar el suelo. Tanto el ángulo de orientación (yaw) como el de inclinación (pitch) se calculan con la misma lógica empleada en el carro, pero

apuntando hacia el waypoint anterior, e incorporando interpolación suave para que las curvas se aprecien de forma natural.

La jerarquía de la nave se construye a partir de la matriz `modelaux`, que contiene la posición y rotación del cuerpo principal. Las cinco piezas que conforman la nave (cuerpo, ala izquierda, ala derecha, hélice izquierda y hélice derecha) se renderizan tomando `modelaux` como matriz padre, lo que asegura que cuando la nave se desplaza o gira, todas sus partes la siguen como un conjunto coherente. Cada pieza dispone además de su propia transformación local para realizar su animación independiente.

El aleteo de las alas se obtiene mediante la función `OscilarEntre(naveAlaAtras, naveAlaFrente, naveOscAla)`, que produce una oscilación entre -5 y 60 grados utilizando una función seno. El ala izquierda emplea el ángulo positivo y el ala derecha el negativo, lo que genera un aleteo simétrico característico de un movimiento aerodinámico.

Las dos hélices giran sobre el eje X mediante la expresión $\text{naveRotHelice} += \text{naveVelHelice} * \text{deltaTime}$. La velocidad asignada (7800 grados por segundo) reproduce un giro continuo similar al de hélices reales en operación. Gracias al ajuste previo del origen de cada hélice realizado en Blender, ambas piezas rotan sobre su propio eje sin desplazarse de la posición que ocupan en el cuerpo de la nave.

El faro de la nave corresponde al `spotLights[3]`, un `SpotLight` adicional que se actualiza en cada frame mediante el método `setFlash`. Su posición se calcula como la posición de la nave desplazada 2.2 unidades hacia adelante y ligeramente hacia abajo. Su dirección apunta principalmente hacia el suelo con una pequeña componente horizontal, de modo que el cono de luz se proyecte sobre la pista justo debajo de la nave durante todo el recorrido.

Para el aterrizaje, cuando `naveWpActual` alcanza el valor cero, se activa la bandera `naveAterrizando`. En ese momento la nave queda fija en las coordenadas X y Z del primer waypoint, y la altura Y se interpola desde el valor `naveAlturaVuelo` (20) hasta `naveAlturaAterrizada` (1.5) mediante la variable `naveAterrizajeT` que recorre el rango de 0 a 1. Al concluir esta interpolación se activa `naveAterrizada = true` y la animación queda detenida de manera definitiva. Cabe destacar que la tecla R únicamente reinicia el carro y nunca afecta a las variables de la nave, lo que

garantiza el cumplimiento del requisito establecido en el enunciado: la animación de la nave no puede repetirse durante la ejecución del programa.

Inicialización y variables de estado de la nave:

```
// Variables principales
float naveAlturaVuelo = 20.0f;
float naveAlturaAterrizada = 1.5f;
float naveVelHelice = 7800.0f;
float naveAlaFrente = 60.0f;
float naveAlaAtras = -5.0f;
bool naveAterrizando = false;
bool naveAterrizada = false;

// Inicio en el ultimo waypoint (sentido contrario)
naveWpActual = (int)pistaWaypoints.size() - 1;
navePos = pistaWaypoints[naveWpActual] + glm::vec3(0.0f, naveAlturaVuelo, 0.0f);
```

Lógica de vuelo en sentido contrario y aterrizaje:

```
if (!naveAterrizada)
{
    if (!naveAterrizando)
    {
        // Va hacia el waypoint anterior (sentido inverso)
        glm::vec3 wpDesde = pistaWaypoints[naveWpActual];
        glm::vec3 wpHacia = pistaWaypoints[naveWpActual - 1];
        glm::vec3 segmento = wpHacia - wpDesde;

        naveT += (naveVelAvance * deltaTime) / glm::length(segmento);

        if (naveT >= 1.0f)
        {
            naveWpActual--;
            if (naveWpActual <= 0)
                naveAterrizando = true;
        }

        // Posicion sobre la pista + altura de vuelo
        navePos = wpDesde + segmento * naveT + glm::vec3(0.0f, naveAlturaVuelo, 0.0f);
    }
    else
    {
        // Fase de aterrizaje
        naveAterrizajeT += deltaTime * 0.45f;
        if (naveAterrizajeT >= 1.0f) naveAterrizada = true;
        float yIni = pistaWaypoints[0].y + naveAlturaVuelo;
        float yFin = pistaWaypoints[0].y + naveAlturaAterrizada;
        navePos.y = yIni + (yFin - yIni) * naveAterrizajeT;
    }
}
```

Jerarquía y animación de las piezas (alas y hélices):

```

// Matriz padre: posicion y orientacion del cuerpo
model = glm::translate(model, navePos);
model = glm::rotate(model, naveYaw * toRadians, glm::vec3(0,1,0));
modelaux = model; // padre

// Animaciones independientes
naveRotHelice += naveVelHelice * deltaTime;
naveOscAla += naveVelAla * deltaTime;
naveAngAla = OscilarEntre(naveAlaAtras, naveAlaFrente, naveOscAla);

// Cuerpo (hereda de modelaux)
Nave_M.RenderModel();

// Ala izquierda con aleteo
model = modelaux;
model = glm::rotate(model, naveAngAla * toRadians, glm::vec3(0,1,0));
Ala_M.RenderModel();

// Ala derecha con aleteo opuesto (simetria)
model = modelaux;
model = glm::rotate(model, -naveAngAla * toRadians, glm::vec3(0,1,0));
Ala2_M.RenderModel();

// Helice izquierda girando sobre eje X
model = modelaux;
model = glm::rotate(model, naveRotHelice * toRadians, glm::vec3(1,0,0));
HeliceIzq_M.RenderModel();

// Helice derecha girando sobre eje X
model = modelaux;
model = glm::rotate(model, naveRotHelice * toRadians, glm::vec3(1,0,0));
HeliceDer_M.RenderModel();

```



Figura 7. Nave en el último waypoint de la pista, posición desde la cual inicia su recorrido en sentido contrario hacia el punto inicial.



Figura 8. Nave sobrevolando la pista con las hélices girando sobre el eje X y las alas en pleno aleteo simétrico, manteniéndose a una altura constante por encima del recorrido.



Figura 9. Faro de la nave proyectando un cono de luz sobre la pista durante el vuelo. La posición y dirección de la luz se actualizan en cada frame mediante el método SetFlash.

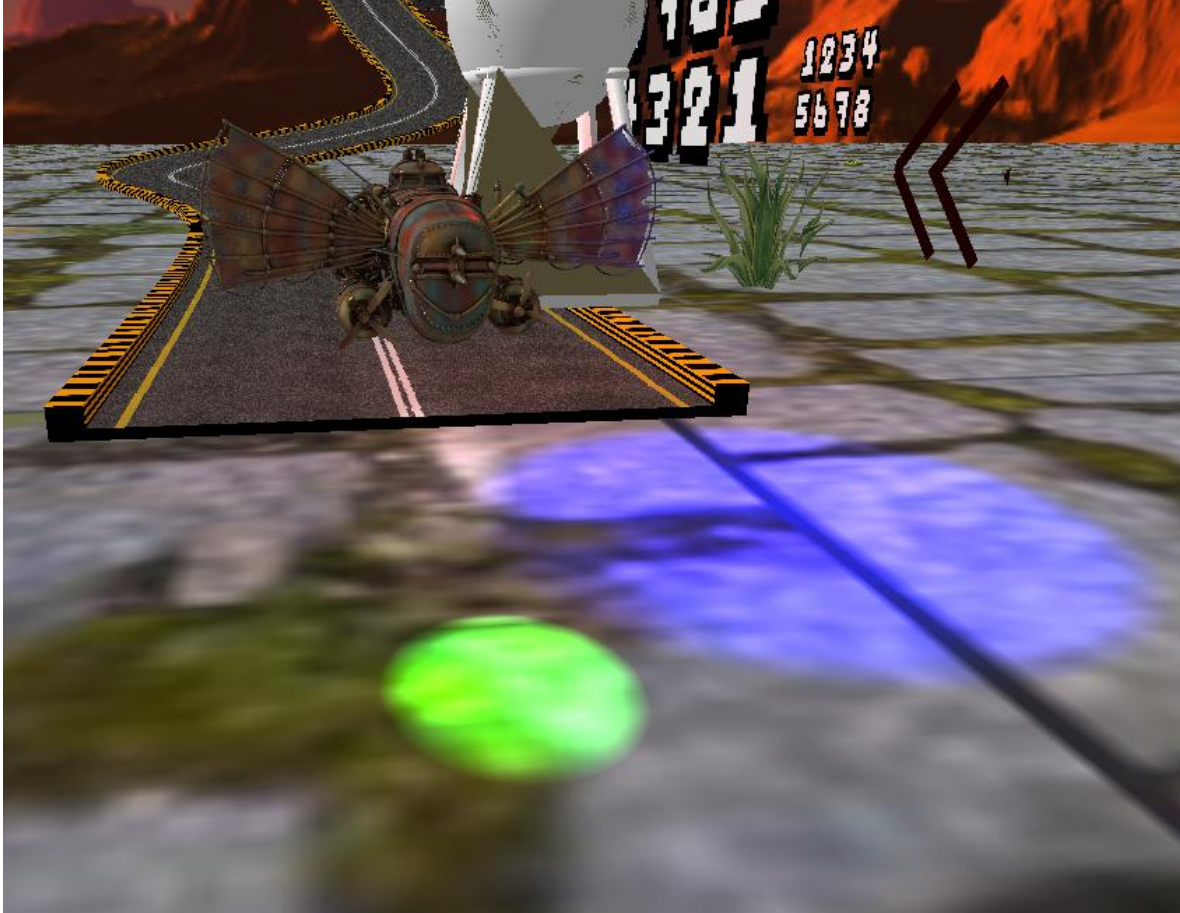


Figura 10. Aterrizaje de la nave junto al primer waypoint de la pista. La altura desciende de manera interpolada desde la altura de vuelo hasta la altura final, momento en que la animación se da por terminada y no puede repetirse.

Teclas utilizadas en el programa:

El programa responde a las siguientes teclas durante su ejecución: WASD permite desplazar la cámara por la escena; el mouse controla la orientación de la cámara; la tecla R reinicia la animación del carro al primer waypoint; la tecla C activa o desactiva el modo de captura de waypoints; la tecla P captura un punto de la pista cuando el modo de captura está activo; y la tecla L permite limpiar la lista de puntos capturados durante dicho modo.

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla.

Uno de los primeros problemas detectados fue que las animaciones iniciales del carro y de la nave se ejecutaban a una velocidad excesivamente alta, debido a que los valores de velocidad se aplicaban directamente sin considerar el tiempo transcurrido entre frames. La solución consistió en multiplicar todas las velocidades por la variable `deltaTime`, lo que garantiza un movimiento consistente independientemente del framerate del equipo en que se ejecute el programa.

Otro problema relevante fue que las llantas del carro giraban como trompos en lugar de rodar correctamente conforme avanzaba el vehículo. Esto se debía a que el eje de rotación se encontraba mal asignado, definido sobre el eje Z en lugar del eje X, lo que producía un giro lateral incoherente con la dirección de avance. La corrección consistió en cambiar el eje de rotación al X e invertir el signo de la variable (`-rotllanta`) para que el sentido de giro coincidiera visualmente con el desplazamiento del carro.

El modelo de la nave importado desde Blender presentaba la parte trasera apuntando hacia adelante, por lo que al desplazarse durante la animación parecía moverse en reversa. Este problema se resolvió aplicando una rotación fija de 180 grados sobre el eje Y antes del render, lo que orientó correctamente la nave sin alterar el resto de la jerarquía de transformaciones.

Las hélices y las alas inicialmente describían trayectorias circulares amplias alrededor de un punto que no correspondía a su centro geométrico, en lugar de rotar sobre su propio eje. La causa de este comportamiento era que al exportar las piezas desde Blender, el origen de cada objeto se mantenía en el centro de la escena (0, 0, 0) y no en el centro de la pieza. Al aplicar una rotación, el modelo giraba alrededor de ese punto externo, generando un desplazamiento amplio en lugar de un giro local. La solución consistió en seleccionar cada pieza dentro de Blender y aplicar la operación `Object → Set Origin → Origin to Geometry`, lo que reposiciona el pivote justo en el centro de la malla, para posteriormente reexportar los archivos `.obj` con el origen ya corregido.

Al implementar el modo de captura de waypoints, las coordenadas registradas inicialmente presentaban valores demasiado altos en el eje Y, dado que la cámara se encontraba elevada respecto al piso de la pista. La solución consistió en restar 1.5 unidades a la altura de la cámara antes de almacenar el punto, de modo que las

coordenadas quedaran al nivel del piso y no a la altura de la vista del observador. Adicionalmente, mientras se conducía con la cámara durante la captura, el carro aparecía en pantalla con la cajuela orientada hacia adelante; este detalle se corrigió sumando 180 grados al ángulo yaw del carro durante el modo de captura, garantizando así que el frente del vehículo apuntara hacia la dirección de la cámara.

El faro de la nave no se proyectaba en la escena pese a estar correctamente definido en el código. El problema se localizó en una inconsistencia entre las constantes MAX_SPOT_LIGHTS declaradas en el código de C++ y en el fragment shader: la primera tenía un valor mayor que la segunda, por lo que el shader únicamente procesaba un número limitado de spotlights y la cuarta luz, correspondiente al faro de la nave, quedaba excluida del cálculo. La solución consistió en igualar el valor de la constante en ambos archivos, asegurando que el shader reservara espacio suficiente para todas las spotlights utilizadas por el programa.

Cuando la trayectoria del carro pasaba por dos waypoints muy próximos entre sí, la longitud del segmento intermedio resultaba prácticamente nula, lo que ocasionaba divisiones entre valores muy pequeños y, en consecuencia, saltos abruptos en la posición del vehículo. Para resolver esta situación se incorporó una validación condicional con la sentencia `if (longitudSeg < 0.001f)`, la cual permite avanzar al siguiente waypoint sin intentar interpolar a lo largo de un segmento de longitud insignificante.

Las transiciones entre waypoints inicialmente resultaban bruscas, ya que el carro modificaba su orientación de forma instantánea al pasar de un segmento al siguiente. Esta limitación se resolvió mediante la implementación de la función `InterpolarYaw`, que suaviza el giro y contempla el wrap-around del rango -180 a 180 grados para que el vehículo siempre realice la rotación por el lado más corto. Un tratamiento equivalente se aplicó al ángulo de inclinación (pitch) por medio de la función `InterpolarSuave`.

3.- Conclusión:

El desarrollo de esta práctica permitió integrar diversos conceptos fundamentales de la computación gráfica en un solo proyecto: animación basada en waypoints, jerarquía de modelos, interpolación suave de ángulos para lograr un movimiento natural, iluminación dinámica mediante faros que siguen objetos en movimiento y

preparación previa de modelos en Blender. La técnica de waypoints demostró ser una solución sólida para que un objeto siga una trayectoria compleja sin necesidad de codificar lógica específica para cada curva o rampa. La jerarquía implementada en la nave evidenció que, mediante una matriz padre y matrices hijas, es posible animar piezas independientes —como las alas con su aleteo y las hélices con su giro continuo— sin perder la coherencia con el cuerpo principal. Tanto el faro frontal del carro como el faro de la nave reforzaron el concepto de que las luces no necesariamente son estáticas: pueden actualizarse en cada frame mediante el método SetFlash para que su posición y dirección se desplacen junto con un objeto en movimiento. El trabajo previo realizado en Blender, particularmente el ajuste del origen de cada pieza, resultó determinante para que las hélices y las alas rotaran correctamente sobre su propio eje en lugar de comportarse como objetos descontrolados. En conjunto, los dos ejercicios transformaron una escena con elementos estáticos en una escena con movimiento coordinado, iluminación dinámica y comportamiento visualmente coherente, cumpliendo de manera satisfactoria con los requisitos planteados en el enunciado de la práctica.

Bibliografía:

- Shreiner, D., Sellers, G., Kessenich, J., & Licea-Kane, B. (2013). OpenGL Programming Guide: The Official Guide to Learning OpenGL, Version 4.3 (8th ed.). Addison-Wesley Professional.
- Blender Foundation. (2025). Blender 4.x Reference Manual: Modeling, Materials and Export. <https://docs.blender.org>
- The Khronos Group. (2024). OpenGL 4.6 Core Profile Specification. <https://registry.khronos.org/OpenGL/specs/gl/glspec46.core.pdf>